

ADVANCED NUMERICAL ANALYSIS

FINAL PROJECT GUIDELINES

Winter 2026

Prof. Chowdhury

For the final project in Math 212, students are required to investigate an advanced numerical method extending beyond the standard syllabus (or substantially extend one we covered), formalize its analytic properties, support their claims with computational evidence, and implement the algorithm to analyze a suitable test problem. You will produce:

- an expository paper (typically 6–10 pages),
- reproducible computational evidence in a notebook (.ipynb or .nb), and
- a 10–15-minute presentation with slides.

Alternatively, you could apply an advanced numerical technique to an appropriate problem in the physical sciences, data science, finance, engineering, etc., **provided you explicitly establish the underlying numerical theory**. All such proposals must be vetted by Dr. Chowdhury to ensure the mathematical depth meets the standards of this course.

§1 Timeline (end of Week 7 through Week 10)

- (a) **Topic proposal (Week 8, Friday 5pm)**. Email me a short proposal (a few sentences) describing:
- your topic and goal(s),
 - what you will prove (a theorem-level statement), and
 - what computational experiments you expect to run.
 - a preliminary bibliography (at least two serious sources; see below).
- (b) **Optional Draft check-in (Week 9, Friday 11:59 pm)**. If you want some early feedback, submit a rough but complete draft:
- a draft PDF with placeholders allowed (e.g. figures not final), and
 - a working notebook that runs end-to-end on at least one test case.
- (c) **Final submission (Week 10, Friday 11:59pm)**. Upload the final paper PDF, final notebook, slides, and presentation recording link/file (see below).

§2 Guidelines

2.1 Sources

You should use at least two serious sources (books, scholarly articles, or high-quality lecture notes). Do not rely on a general website as a main source of mathematical claims. Web searches are fine for idea generation, but any theorem/algorithm statement you use should be traceable to a reliable source.

Your paper should list **all sources** used (any standard citation style is fine). Citations within the text should follow the author-date convention. For example:

- **Single author:** The stability region is constrained by the spectral radius of the iteration matrix (Higham, 2002).
- **Multiple authors:** Trefethen and Bau (1997) demonstrate that the QR factorization via Householder reflections is backward stable.
- **More than two authors:** Tyson et al. (2004) found that something interesting occurred.

If you copy a figure, cite the source in the caption.

2.2 Generative AI

You may use GenAI for coding if you cite it, but the mathematical narrative and validation must be yours.

2.3 Expository paper (6–10 pages, not counting appendix/code)

The paper should be written in an academic style and should read like a short expository article. A typical strong structure is:

- **Introduction and problem statement.** What is the computational problem? Why is it interesting? What are the main claims?
- **Mathematical setup.** Precise definitions, assumptions, norms/inner products if relevant, and what is being approximated.
- **Methodology.** Describe the algorithm at an advanced level (including pseudocode if helpful). Make clear what inputs/outputs are.
- **Theorem + proof.** State and prove at least one theorem. The theorem should be mathematically non-trivial (not just algebraic manipulation) and genuinely connected to your method (e.g. convergence, stability, error bound, conditioning, complexity, existence/uniqueness of an object the method uses).
- **Numerical analysis discussion.** Address error/stability/convergence/conditioning issues meaningfully. This should not be a single paragraph; it should actually engage with what can go wrong and why your design choices avoid it (or quantify the failure).
- **Computational evidence.** Summarize experiments and interpret them (do not just copy/paste plots).
- **Conclusion.** What did you learn? What would you improve with more time?

Keep the main body readable, put the code in the notebook (and/or an appendix).

2.4 Notebook (reproducible computational evidence)

Submit a single .ipynb or .nb notebook that:

- can be run without any edits (set default values for input, but the user should be allowed to change it when relevant),
- clearly separates **implementation** from **experiments**,
- includes well-labeled figures and tables, and

- contains enough commentary that a reader can understand what you are doing and why.

Your experiments should support your theoretical story (e.g. verify an order of accuracy on a log–log plot, illustrate stability breakdown, demonstrate complexity scaling).

2.5 Recorded presentation (10–15 minutes)

There will be no in-class project presentations. Instead, you will record a 10–15 minute presentation with slides. **Assume your audience is your peers, not your professors.**

Format

Record your screen while presenting slides with voice-over (Zoom/Teams/Panopto/etc. are all fine). The recording should be playable without special permissions. Submit:

- the slides (.pdf preferred), and
- either an .mp4 file or a link (unlisted/private is fine as long as staff can view).

Alternatively, you may record yourself giving the presentation using a blackboard. But I will expect a high-quality product.

What to include

You will not have time for full proof details. Include:

- the problem and motivation,
- the main idea of the algorithm,
- a clearly stated theorem and the proof strategy (key ideas),
- the key numerical analysis takeaway (stability/error/efficiency),
- 1–3 informative computational results (plots/tables) and interpretation.

Slides

Keep the number of slides around 5-10. Avoid dense text. Your audience should learn mainly by listening, but the slides should contain enough structure and notation to follow the argument.

§3 Grading Rubric (100 points)

The project rubric (100 points total) is detailed below. Full credit requires rigorous, professional-level execution. Partial credit is awarded based on the severity of errors, omissions, or lack of mathematical rigor.

Criterion	Pts	Full Credit	Partial / Reduced Credit
Mathematical Content and Rigor (60 points)			

Criterion	Pts	Full Credit	Partial / Reduced Credit
Problem formulation, scope, and goals	10	Clear, rigorous statement of the problem, well-defined scope, and specific goals.	Vague formulation, scope too broad/narrow, or missing key mathematical definitions.
Methodology described at an advanced level	10	Advanced-level description of the numerical method appropriate for Math 212.	Method described superficially, lacking advanced context, or resembles introductory material.
Theorem + complete proof	22	States a relevant, non-trivial theorem with a completely rigorous, step-by-step mathematical proof.	Minor logical gaps, missing steps, or theorem is too trivial. (0 pts for invalid/unverified AI proofs).
Numerical analysis	18	Thorough analysis of error, stability, convergence, or conditioning using advanced techniques.	Incomplete analysis, relies entirely on empirical observation without theory, or misses key issues.
Expository Paper Quality (15 points)			
Organization and narrative	6	Logical flow, clear headings, engaging narrative structure suitable for a math paper.	Disjointed sections, poor transitions, or difficult to follow the main argument.
Math correctness, notation, citations	6	Flawless $\text{\LaTeX}$ notation, mathematically sound text, proper and rigorous citations.	Sloppy notation, typos in equations, or missing/improper citations (including hallucinated AI citations).
Figures/tables and interpretation	3	Well-labeled, high-quality visuals that directly support the text and are clearly interpreted.	Missing labels, poorly scaled, or disconnected from the narrative text.
Computational Evidence via Notebook(s) (10 points)			
Verification / experimental design	6	Robust experimental design that explicitly verifies the theoretical claims (e.g., convergence rates).	Weak testing, fails to verify theoretical bounds, or only tests trivial edge cases.
Reproducibility and notebook quality	4	Clean, well-commented code that runs perfectly from top to bottom.	Messy code, requires substantial manual intervention to run, or lacks explanatory comments.
Presentation (slides) (15 points)			
Structure, timing, and narrative	5	Fits strictly within 10–15 mins, excellent pacing, clear beginning/middle/end.	Rushed, goes significantly over/under time, or lacks a coherent narrative arc.

Criterion	Pts	Full Credit	Partial / Reduced Credit
Mathematical communication	5	Explains complex math intuitively to peers, focuses on insights rather than tedious algebra.	Gets bogged down in algebra, fails to explain intuition, or reads equations directly off slides.
Visual communication	5	Slides are uncluttered, visual aids enhance understanding, professional design.	Text-heavy slides, poor contrast, or confusing/missing graphics.

§4 Suggested Topics

You may propose your own topic. The list below is meant to be idea-generating; pick something with a theorem you can realistically prove and experiments you can realistically run in the available time.

4.1 Numerical Linear Algebra

- (a) Krylov methods (GMRES/MINRES) and the role of preconditioning.
- (b) Conditioning & backward error in linear solves
- (c) Eigenvalue algorithms (QR / Rayleigh quotient / Arnoldi-Lanczos)
- (d) Randomized SVD or randomized least squares; probabilistic error bounds, compare against deterministic algorithms on large datasets.

4.2 Approximation and Interpolation

- (e) Barycentric Lagrange interpolation and stability; compare to naive Vandermonde approaches.
- (f) Chebyshev expansions + FFT-based polynomial evaluation/differentiation; spectral accuracy.
- (g) Padé Approximants: extend polynomial approximation by investigating rational function approximation; existence and uniqueness, analyze advantage over Taylor polynomials

4.3 Quadrature

- (h) Clenshaw–Curtis quadrature (and why it competes with Gauss in practice).
- (i) Gauss–Kronrod / embedded quadrature rules and adaptive error control.
- (j) High-dimensional integration: Monte Carlo vs quasi-Monte Carlo

4.4 ODEs

- (k) Adaptive Runge–Kutta pairs (e.g. Dormand–Prince): local error estimation and step-size control.
- (l) Stiff ODEs: A-stability/L-stability, implicit methods (Backward Euler, trapezoid rule, BDF), and stability regions.

- (m) Numerical Methods for Stochastic Differential Equations (SDEs) Euler–Maruyama / Milstein; weak vs strong convergence

4.5 PDE-flavored projects

- (n) Finite differences for Poisson’s equation with an iterative solver (Jacobi/Gauss–Seidel/SOR) and a convergence theorem in a suitable norm.
- (o) Multigrid idea on a model elliptic problem (even a two-grid analysis can be a good theorem target).
- (p) Spectral collocation with Chebyshev differentiation matrices; spectral convergence
- (q) Finite Element Method for Poisson: weak form, Lax–Milgram, Galerkin orthogonality